Prompting Techniques

Zero-Shot, Few-Shot, Chain-of-Thought & More

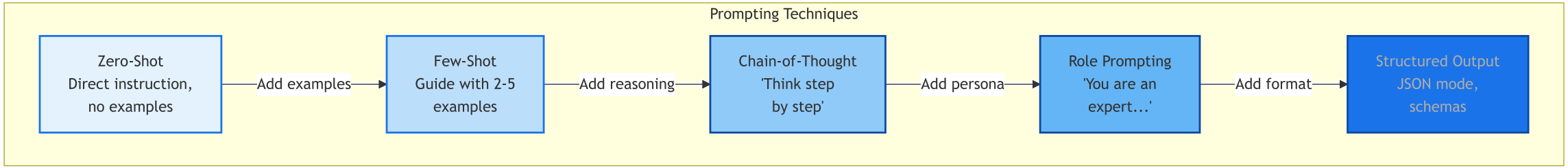
Day 2 - Session 2

KavinSchool

Session 2: What We'll Cover

1. **Zero-shot** prompting: direct instruction
2. **Few-shot** prompting: guiding with examples
3. **Chain-of-Thought (CoT):** step-by-step reasoning
4. **Role prompting:** expert personas
5. **Structured output:** JSON mode & schemas
6. Delimiters, headers, and formatting tricks

Overview of Techniques



Zero-Shot Prompting

Just Ask, No Examples Needed

KavinSchool

Zero-Shot: Direct Instruction

Give the model a task with **no examples**. The model relies entirely on its training.

```
messages = [
    {"role": "system", "content": "Classify the sentiment as POSITIVE, "
                                  "NEGATIVE, or NEUTRAL."},
    {"role": "user", "content": "The new update completely broke my workflow."},
]
# Output: "NEGATIVE"
```

When to use:

- Simple, well-defined tasks
- Classification with clear categories
- The model already "knows" the task pattern

Zero-Shot: Direct Instruction

When it fails:

- Ambiguous tasks
- Domain-specific classification
- When your labels don't match common usage

Zero-Shot: More Examples

Summarization:

```
system = "Summarize the following text in exactly 3 bullet points."
```

Code Generation:

```
system = "Generate a Python function based on the description. Include type hints and a docstring."
```

Translation:

```
system = "Translate the following English text to Japanese. Preserve technical terms in English."
```

> Zero-shot works surprisingly well for standard NLP tasks. Always try it first before adding complexity.

Few-Shot Prompting

Teach by Example

KavinSchool

Few-Shot: Adding Examples

Provide 2-5 examples to **teach the model your pattern**.

```
messages = [
    {"role": "system", "content": "Classify the sentiment."},
    {"role": "user", "content": "This product is amazing! Best purchase ever."},
    {"role": "assistant", "content": "POSITIVE"},
    {"role": "user", "content": "Terrible quality. Broke after one day."},
    {"role": "assistant", "content": "NEGATIVE"},
    {"role": "user", "content": "It works as expected. Nothing special."},
    {"role": "assistant", "content": "NEUTRAL"},
    # Now classify this one:
    {"role": "user", "content": "The battery life could be better but overall I like it."},
]
# Output: "POSITIVE"
```

Few-Shot: Why It Works

Aspect	Zero-Shot	Few-Shot
Examples provided	None	2-5
Model learns from	Training data only	Training + your examples
Consistency	Variable	Much more consistent
Token cost	Lower	Higher (examples use tokens)
Best for	Standard tasks	Custom tasks, domain-specific

Key insight: Few-shot examples teach the model your **format**, **style**, and **edge case handling**.

Few-Shot: In-Prompt vs Message-Based

Option A: Examples in the user message

```
user = """Classify the intent of customer messages:

"I can't log in" → account_access
"How much does it cost?" → pricing
"The app keeps crashing" → bug_report

Classify: "Can I upgrade my plan?"
"""
```

Option B: Examples as message pairs (shown on previous slide)

> Both work. Message-based is better for longer examples. In-prompt is more compact.

Few-Shot: Best Practices

1. **Use 3-5 examples.** Enough to show the pattern, not so many that you waste tokens
2. **Cover edge cases.** Include tricky examples the model might get wrong
3. **Balance categories.** Don't show 4 positive and 1 negative
4. **Match the real distribution.** If most inputs are technical, most examples should be too
5. **Order matters.** Put the most representative examples first and last (recency + primacy)

```
# ❌ Bad: all positive examples
examples = ["Great!", "Awesome!", "Love it!", "Perfect!", "Amazing!"]

# ✅ Good: balanced with edge cases
examples = ["Great!", "Terrible.", "It's okay I guess.", "Not bad but not great.", "Awful!"]
```

Chain-of-Thought (CoT)

"Think Step by Step"

KavinSchool

Chain-of-Thought: Making LLMs Reason

Instead of asking for the final answer directly, ask the model to **show its reasoning**.

Without CoT:

```
Q: A store has 15 apples. 8 are sold in the morning,
   then 3 more are delivered. How many apples remain?
A: 10  ✅ (sometimes right, sometimes wrong)
```

With CoT:

```
Q: A store has 15 apples. 8 are sold in the morning,
   then 3 more are delivered. How many apples remain?
   Think step by step.
A: Starting count: 15 apples
   After selling 8: 15 - 8 = 7 apples
   After delivery of 3: 7 + 3 = 10 apples
   Final answer: 10 apples ✅ (much more reliable)
```

CoT: When to Use It

Task Type	Without CoT	With CoT
Simple classification	Works fine	Overkill
Multi-step math	Unreliable	Much better
Code debugging	Misses steps	Catches more bugs
Logical reasoning	Often wrong	Significantly better
Decision making	Jumps to conclusion	Considers tradeoffs

The magic phrase: `"Think step by step"` or `"Let's work through this step by step."`

CoT: Practical Example for Developers

```
messages = [
    {"role": "system", "content":
        "You are a code review assistant. When analyzing code:\n"
        "1. First, describe what the code does\n"
        "2. Then, identify potential issues step by step\n"
        "3. For each issue, explain WHY it's a problem\n"
        "4. Finally, suggest a fix\n\n"
        "Show your reasoning at each step."},
    {"role": "user", "content": f"Review this code:\n\n```python\n{code}\n```"},
]
```

> CoT prompting improved GPT-4's performance on the GSM8K math benchmark from 80% to 95%.

CoT Variants

Standard CoT: "Think step by step"

Self-consistency: Run the same CoT prompt N times, take the majority answer

```
# Run 5 times, take the most common answer
answers = [get_completion(messages) for _ in range(5)]
final = most_common(answers)
```

Plan-and-Execute: "First, outline your plan. Then execute each step."

Structured CoT: Force the reasoning into a specific format

```
system = """For each question, respond in this format:
ANALYSIS: [your step-by-step reasoning]
CONCLUSION: [your final answer]
CONFIDENCE: [HIGH/MEDIUM/LOW]"""
```

Role Prompting

"You Are an Expert..."

KavinSchool

Role Prompting: Shaping Model Behavior

Assigning a **role** or **persona** changes the model's response style, depth, and focus.

```
# Generic
system = "Answer the question."
# → Gives a textbook answer

# With role
system = "You are a senior Python developer with 15 years of experience. You have deep expertise in Django, PostgreSQL, and distributed systems. You value clean code, comprehensive testing, and clear documentation."
# → Gives practical, opinionated, production-focused answer
```

Role Prompting: Examples

Role	Effect on Response
"Senior security engineer"	Focuses on vulnerabilities, OWASP, threat models
"Junior developer asking questions"	Simpler language, more examples
"Technical writer"	Structured, clear, well-organized
"Code reviewer at Google"	Strict, follows style guides, nit-picky
"Friendly Python tutor"	Patient, encouraging, step-by-step
"DevOps engineer"	Infrastructure-focused, mentions CI/CD, monitoring

Combining Techniques

You can combine **all** techniques in a single prompt:

```
messages = [
    # ROLE PROMPTING
    {"role": "system", "content":
        "You are a senior Python developer specializing in API design. "
        "Think step by step when analyzing code. "  # CoT
        "Respond as JSON with fields: issues[], suggestions[], score."},  # Format

    # FEW-SHOT EXAMPLES
    {"role": "user", "content": "Review: def get_user(id): ..."},
    {"role": "assistant", "content": '{"issues": ["no type hints"], '
                                     '"suggestions": ["add -> User"], '
                                     '"score": 7}'},

    # ACTUAL REQUEST
    {"role": "user", "content": f"Review:\n```python\n{code}\n```"},
]
```

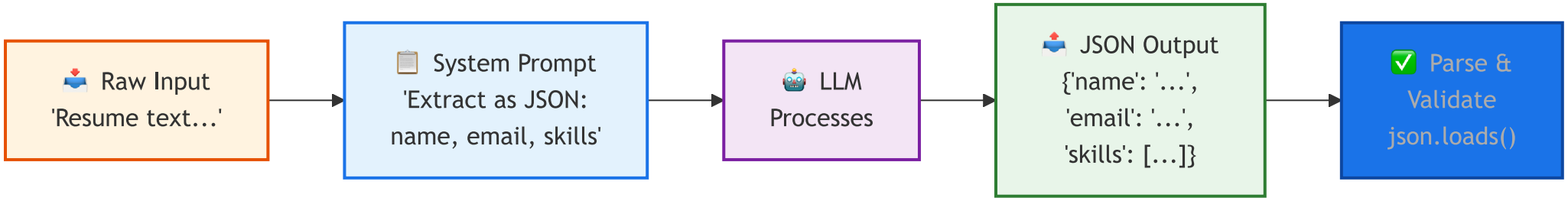
Structured Output

Getting Predictable, Parseable Responses

KavinSchool

Structured Output: Why It Matters

LLMs generate **free-form text** by default. For production systems, you need **parseable, predictable output**.



Strategy 1: Prompt-Based JSON

The simplest approach: ask for JSON in the prompt.

```
system = """Extract contact information from the text.
Respond with ONLY valid JSON, no other text:

{
    "name": "full name",
    "email": "email address or null",
    "phone": "phone number or null",
    "company": "company name or null"
}"""

user = "Hi, I'm Alex Rivera from TechCorp. Email me at alex@techcorp.com."
```

Pros: Works with any provider, simple

Cons: Model might add markdown fences, extra text, or invalid JSON

Strategy 2: JSON Mode (OpenAI)

OpenAI supports a `response_format` parameter:

```
import litellm

response = litellm.completion(
    model="gpt-4o",
    messages=[
        {"role": "system", "content": "Extract contacts as JSON."},
        {"role": "user", "content": text},
    ],
    response_format={"type": "json_object"},
)
# Guaranteed valid JSON (no markdown fences, no extra text)
result = json.loads(response.choices[0].message.content)
```

> **Important:** You must still describe the JSON schema in your prompt. JSON mode only guarantees valid JSON, not a specific structure.

Delimiters and Formatting Tricks

Use delimiters to **clearly separate** different parts of your prompt:

```
user = f"""
### INSTRUCTIONS
Summarize the document below in 3 bullet points.

### DOCUMENT
---
{document_text}
---

### OUTPUT FORMAT
- Bullet 1: [key finding]
- Bullet 2: [key finding]
- Bullet 3: [key finding]
"""
```

Delimiters and Formatting Tricks

Common delimiters: `---`, `"""`, `###`, `===`, `<tags>`, `[SECTION]`

> Delimiters reduce ambiguity. The model clearly knows where instructions end and data begins.

Session 2: Key Takeaways

1. **Zero-shot works for simple tasks.** Always try it first
2. **Few-shot teaches your pattern.** 3-5 balanced examples dramatically improve consistency
3. **Chain-of-Thought for reasoning.** "Think step by step" improves accuracy significantly
4. **Role prompting shapes tone and depth.** Match the persona to the task
5. **Structured output is essential for production.** JSON mode or prompt-based schemas
6. **Combine techniques freely.** Role + CoT + few-shot + JSON = effective prompts

Next: Session 3, API Parameters. Temperature, top_p, streaming, and more!

Questions?

Session 2 Complete. Lunch Break!

KavinSchool